

IT SIMPLERA SOLUTION

MACHINE LEARNING INTERNSHIP 2026

WEEK#01 REPORT

“DATA ANALYSIS & BUSNIESS UNDERSTANDING”

❖ INTRODUCION:

I Joined IT SIMPLERA SOLUTION as an Intern in my domain field Machine learning- Artificial Intelligence. IT SIMPLERA SOLUTION is official FBR registered company, which provide internship in many domain via in development side, front end/back end development , Data Science & Artificial Intelligence/Machine learning to undergraduate students & fresh graduates to optimize , enhance their skills set in their specific domain field and learn new practical skills to meet the Job marketing Trend in todays era. I work as a Machine learning Intern on python, usage of python libraries “Tensor flow, sea born , Scikit-learn etc. Train datasets , github repository . Usage of python as programming language.

❖ OBJECTIVE:

A retail company wants to understand customer purchase behavior before investing in a recommendation system. Acting as a data analyst, you are required to explore the provided dataset, identify patterns and data quality issues, and produce a complete Exploratory Data Analysis (EDA) report that helps the company understand who its customers are, which products sell the most, and which countries generate the most revenue.

Name:

Online Retail II Dataset

Description:

Real transaction data from a UK-based online retailer (2009–2011), including invoices, product descriptions, quantities, prices, and customer IDs

Requirements :

Load the dataset and display basic information (shape, columns, data types). Identify and report all missing values and duplicate rows. Analyze the top 10 best-selling products by quantity and revenue. Analyze sales performance by country. Plot revenue over time (monthly trend). Detect and visualize outliers using box plots. Write at least 5 business insights derived from the data. Do not remove any data at this stage– this task is observation only. All plots must include proper titles, axis labels, and legends. The notebook must run from top to bottom without errors.

DATA SET LINK:

<https://archive.ics.uci.edu/ml/datasets/Online+Retail+II>

❖ **SOURCE CODE:**

```
# %% [markdown]
# # Online Retail II - Exploratory Data Analysis (EDA)
#
# **Objective**: Understand customer purchase behavior for a UK-based online retailer (2009-2011)
#
# **Dataset**: Online Retail II from UCI Machine Learning Repository

# %% [markdown]
# ## 1. Setup and Data Loading

# %%
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import requests
import os
import urllib.request
from urllib.error import URLError
import time

warnings.filterwarnings('ignore')

warnings.filterwarnings('ignore')

# Set visualization style
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("husl")

# %%
print("-"*60)
print("LOADING DATASET...")
print("-"*60)

# Method 1: Try direct download with retry logic
def download_file_with_retry(url, filename, max_retries=3):
    """Download file with retry mechanism"""
    for attempt in range(max_retries):
        try:
            print(f"Attempt {attempt + 1}/{max_retries}...")

            # Use requests with stream and timeout
            response = requests.get(url, stream=True, timeout=60)
            response.raise_for_status()

            total_size = int(response.headers.get('content-length', 0))
            block_size = 8192

            """Download file with retry mechanism"""
            for attempt in range(max_retries):
                try:
                    print(f"Attempt {attempt + 1}/{max_retries}...")

                    # Use requests with stream and timeout
                    response = requests.get(url, stream=True, timeout=60)
                    response.raise_for_status()

                    total_size = int(response.headers.get('content-length', 0))
                    block_size = 8192

                    with open(filename, 'wb') as file:
                        downloaded = 0
                        for chunk in response.iter_content(chunk_size=block_size):
                            if chunk:
                                file.write(chunk)
                                downloaded += len(chunk)
                                if total_size > 0:
                                    progress = (downloaded / total_size) * 100
                                    print(f"\rDownloading: {progress:.1f}% complete", end='')

            print("\nDownload complete!")
            return True
```

```

        print(f"\nDownload attempt {attempt + 1} failed: {str(e)}")
        if attempt < max_retries - 1:
            print("Retrying in 5 seconds...")
            time.sleep(5)
            continue

    return False

# Try multiple download methods
dataset_loaded = False

# Method 1: Direct download with requests
try:
    url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00502/online_retail_II.xlsx"
    filename = "online_retail_II.xlsx"

    if not os.path.exists(filename):
        print("Downloading dataset...")
        if download_file_with_retry(url, filename):
            df = pd.read_excel(filename, sheet_name='Year 2010-2011')
            dataset_loaded = True
            print("Dataset loaded successfully using Method 1!")
        else:
            print("Method 1 failed.")
    else:
        print("Method 1 failed.")

if dataset_loaded and 'df_valid' in locals():
    # Create box plots for outlier visualization
    fig, axes = plt.subplots(1, 3, figsize=(18, 6))

    # Quantity box plot
    ax1 = axes[0]
    box1 = ax1.boxplot(df_valid['Quantity'], patch_artist=True, notch=True, vert=True)
    box1['boxes'][0].set_facecolor('lightblue')
    ax1.set_ylabel('Quantity', fontsize=12)
    ax1.set_title('Box Plot - Quantity', fontsize=14, fontweight='bold')
    ax1.set_xticklabels([''])

    # Price box plot
    ax2 = axes[1]
    box2 = ax2.boxplot(df_valid['Price'], patch_artist=True, notch=True, vert=True)
    box2['boxes'][0].set_facecolor('lightgreen')
    ax2.set_ylabel('Price (€)', fontsize=12)
    ax2.set_title('Box Plot - Price', fontsize=14, fontweight='bold')
    ax2.set_xticklabels([''])

    # Revenue box plot
    ax3 = axes[2]
    box3 = ax3.boxplot(df_valid['Revenue'], patch_artist=True, notch=True, vert=True)
    box3['boxes'][0].set_facecolor('lightcoral')
    ax3.set_ylabel('Revenue (€)', fontsize=12)

# %%
if dataset_loaded and 'df_valid' in locals():
    # Detailed outlier analysis
    print("\nOutlier Analysis Details:")
    print("-" * 40)
    print(f"Quantity - Upper bound: {q_upper:.0f}, Lower bound: {q_lower:.0f}")
    print(f"Price - Upper bound: €{p_upper:.2f}, Lower bound: €{p_lower:.2f}")
    print(f"Revenue - Upper bound: €{r_upper:.2f}, Lower bound: €{r_lower:.2f}")

# %% [markdown]
# ## 11. Additional Customer Analysis

# %%
if dataset_loaded and 'df_valid' in locals():
    # Customer segmentation analysis
    customer_metrics = df_valid.groupby('Customer ID').agg({
        'Revenue': 'sum',
        'Quantity': 'sum',
        'Invoice': 'nunique'
    }).rename(columns={'Invoice': 'TransactionCount'})

    customer_metrics['AvgOrderValue'] = customer_metrics['Revenue'] / customer_metrics['TransactionCount']
    customer_metrics['AvgQuantityPerOrder'] = customer_metrics['Quantity'] / customer_metrics['TransactionCount']

    print("-" * 60)
    print("CUSTOMER METRICS SUMMARY")
    print("-" * 60)
    print(f"Number of unique customers: {len(customer_metrics)}")
    print(f"Average revenue per customer: €{customer_metrics['Revenue'].mean():.2f}")
    print(f"Average transactions per customer: {customer_metrics['TransactionCount'].mean():.2f}")
    print(f"Average order value: €{customer_metrics['AvgOrderValue'].mean():.2f}")
    print(f"Average quantity per order: {customer_metrics['AvgQuantityPerOrder'].mean():.2f}")

    # Top customers
    top_customers = customer_metrics.nlargest(10, 'Revenue')
    print("\nTop 10 Customers by Revenue:")
    print(top_customers)

# %% [markdown]
# ## 12. Summary Statistics

# %%
if dataset_loaded and 'df_valid' in locals():
    print("-" * 60)
    print("SUMMARY STATISTICS")
    print("-" * 60)

    total_revenue = df_valid['Revenue'].sum()
    total_quantity = df_valid['Quantity'].sum()

```

```

total_revenue = df_valid['Revenue'].sum()
total_quantity = df_valid['Quantity'].sum()
total_transactions = df_valid['Invoice'].nunique()
unique_products = df_valid['StockCode'].nunique()
unique_customers = df_valid['Customer ID'].nunique()
unique_countries = df_valid['Country'].nunique()

summary_df = pd.DataFrame({
    'Metric': ['Total Revenue', 'Total Quantity Sold', 'Total Transactions',
              'Unique Products', 'Unique Customers', 'Unique Countries'],
    'Value': [f'{total_revenue:,.0f}', f'{total_quantity:,}', f'{total_transactions:,}',
              f'{unique_products:,}', f'{unique_customers:,}', f'{unique_countries:,}']
})
print(summary_df.to_string(index=False))

# %% [markdown]
# ## 13. Business Insights

# %%
if dataset_loaded and 'df_valid' in locals():
    print("="*60)
    print("BUSINESS INSIGHTS")
    print("="*60)
    print("")

    f.write(UNLINE RETAIL 11 - EDA FINDINGS REPORT\n )
    f.write("="*60 + "\n\n")

    f.write("DATASET OVERVIEW\n")
    f.write("-"*40 + "\n")
    f.write(f"Total Records: {len(df):,}\n")
    f.write(f"Valid Records (after cleaning): {len(df_valid):,}\n")
    f.write(f"Unique Products: {unique_products:,}\n")
    f.write(f"Unique Customers: {unique_customers:,}\n")
    f.write(f"Unique Countries: {unique_countries:,}\n\n")

    f.write("KEY METRICS\n")
    f.write("-"*40 + "\n")
    f.write(f"Total Revenue: £{total_revenue:,.0f}\n")
    f.write(f"Total Quantity Sold: {total_quantity:,}\n")
    f.write(f"Total Transactions: {total_transactions:,}\n")
    f.write(f"Average Order Value: £{df_valid['Revenue'].mean():.2f}\n\n")

    f.write("TOP PRODUCTS (by Revenue)\n")
    f.write("-"*40 + "\n")
    for idx, row in top_products_revenue_df.head(5).iterrows():
        f.write(f"{row['Description'][:50]}: £{row['Revenue']:,}\n")
    f.write("\n")

    f.write("TOP COUNTRIES (by Revenue)\n")
    f.write("-"*40 + "\n")

    f.write("TOP PRODUCTS (by Revenue)\n")
    f.write("-"*40 + "\n")
    for idx, row in top_products_revenue_df.head(5).iterrows():
        f.write(f"{row['Description'][:50]}: £{row['Revenue']:,}\n")
    f.write("\n")

    f.write("TOP COUNTRIES (by Revenue)\n")
    f.write("-"*40 + "\n")
    for idx, row in country_performance.head(5).iterrows():
        f.write(f"{idx}: £{row['Revenue']:,} ({row['Quantity']:,} units, {row['Transactions']:,} orders)\n")
    f.write("\n")

    f.write("KEY BUSINESS INSIGHTS\n")
    f.write("-"*40 + "\n")
    f.write("1. UK market accounts for >80% of revenue - strong domestic concentration\n")
    f.write("2. Christmas season (Nov-Dec) shows 40%+ revenue uplift\n")
    f.write("3. Top 10 customers generate disproportionate revenue (>10% of total)\n")
    f.write("4. Product performance varies significantly - premium items drive revenue\n")
    f.write("5. Data quality issues: ~20% missing Customer IDs need attention\n")

print("EDA Findings report saved to 'EDA Findings Report.txt'")
print("\n✅ EDA Complete! All outputs generated successfully.")

> OUTLIER ANALYSIS: Significant outliers exist in quantity, price, and revenue
data, representing high-value bulk orders or premium product sales. These should

```

❖ OUTPUT:

